

Lab04. Chat application.

Cloud Programming (W04IST–SI4527G)

Wojciech Thomas

Spring 2026

1 Learning goals

1. Learn the Chat application.
2. Running simple containers.
3. Create container image.
4. Run container from the image.
5. Orchestrate multiple containers.

2 Prerequisites

Resize the EC2 instance related to your Cloud9 environment for better performance, resize the storage volume, and open ports 3000 and 5000 for the Internet access.

1. Close all Cloud9 tabs, if you have opened any.
2. Find the EC2 instance related to your Cloud9 environment.
3. Change its state to Stopped, if it is running.
4. Change size of the EC2 instance:
 1. Select Actions > Instance settings > Change instance type
 2. Set “New instance type” to `t2.small` (or `t3.small` if `t2.small` is not available).
 3. Click “Change” button.
5. Open security group assigned to Cloud9 EC2 instance (select EC2 instance, then open Security tab at the bottom).
6. Edit inbound rules. **Do not remove or replace** existing rules. **Add** two inbound rules:
 1. Custom TCP - port 5000 - source 0.0.0.0/0 - description: Backend
 2. Custom TCP - port 3000 - source 0.0.0.0/0 - description: FrontendSave security group settings.
7. Select again EC2 instance. At lower part of the screen, select **Storage** tab. Find the volume and click its Volume ID.
8. In Volumes screen, select the volume, and click Action > Modify Volume. Change the size to 15 GB, and click Modify button. Confirm modification. Wait until the visible size of the volume is changed to 15 GB.
9. Start your Cloud9 IDE again. Do not worry about EC2 instance, as it will be started automatically.
10. In Terminal window check the size of the disk: `df -h /`

3 Exercises

3.1 Run web application locally

1. Start SSH agent, and add your private GitHub key (in doubts see the previous lab instruction).
2. Accept the assignment on GitHub classroom, and clone it to Cloud9:

```
$ cd ~/environment
$ git clone git@github.com:pwr-cloudprogramming/<YOUR-REPO>.git
$ cd <YOUR-REPO>
```

3.1.1 Run the backend application

1. Open `backend/` folder and build backend application (you need to add execution permission to `gradle` script to run it directly):

```
$ cd backend
$ chmod u+x ./gradlew
$ ./gradlew clean build
```

2. Run your backend application:

```
$ java -jar build/libs/ChatApp-0.0.1-SNAPSHOT.jar
```

3. Test backend application in the shell:

1. Open a new Terminal window.
2. Run command:

```
$ curl http://<CLOUD-9-PUBLIC-IP>:5000/chat/all?username=test
```

4. Open backend application in browser:

1. Start browser, and in the address bar type:

```
http://<CLOUD-9-PUBLIC-IP>:5000/chat/all?username=test
```

2. As the response you should see: `{"messages":[]}`

3.1.2 Run the frontend application

1. Open the new Terminal window, then open `frontend/` folder and initialize environment:

```
$ cd ../frontend
$ npm install
```

2. Configure backend location in the environment variable: create a new file `.env` in the `frontend/` folder, and paste the following content:

```
PUBLIC_API_BASE_URL=http://<CLOUD-9-PUBLIC-IP>:5000/
```

3. Start the frontend application:

```
$ npm run dev
```

4. Open frontend application in the browser:

1. Start a browser, and in the address bar type: `http://<CLOUD-9-PUBLIC-IP>:3000/`
2. Send some messages.
3. Open the second browser window, and paste the same address. Can you see previous messages?

4 Assignment

1. Prepare `Dockerfile` files for both sides: frontend and backend [1].
2. Build local container images.
3. Start containers manually, for frontend and backend separately.
4. Prepare `compose.yaml` file to start both containers [2]. Put the file in the project folder.
5. Run application using `docker compose`.
6. Prepare the report in the `README.md` file in the project folder. Put the following information inside:
 1. For both frontend and backend:
 1. How to build container images.
 2. How to run container.
 3. What must be configured to do a successful build.
 2. Describe what did you learn.
 3. List all problems encountered.
7. Remember that every significant modification of the project should be committed.
8. Both team members should contribute to the assignment solution.

4.1 Tips

1. For the backend, you can start with `Dockerfile` using pre-build JAR file, as in the first step.
2. As a base image use the container image including Java. Do not install Java manually [3].
3. Finally use multistage image: start with JDK image, to build application from the source, and then copy built JAR into second image with JRE [4].
4. In the multistage process, when building backend image, mount a cache volume for Gradle dependencies, to speed up the build process.

Bibliography

- [1] “Dockerfile overview.” [Online]. Available: <https://docs.docker.com/build/concepts/dockerfile/>.

- [2] “Docker compose quickstart.” [Online]. Available: <https://docs.docker.com/compose/gettingstarted/>.
- [3] “Building best practices.” [Online]. Available: <https://docs.docker.com/build/building/best-practices/>.
- [4] “Multi-stage builds.” [Online]. Available: <https://docs.docker.com/build/building/multi-stage/>.